

**Proyecto Especial**

**Diseño e implementación de un lenguaje**

# Synchro

| Nombre       | Apellido        | Legajo | E-mail                                                                       |
|--------------|-----------------|--------|------------------------------------------------------------------------------|
| Manuel       | Ahumada         | 64.677 | <a href="mailto:maahumada@itba.edu.ar">maahumada@itba.edu.ar</a>             |
| Josefina     | Gonzalez Cornet | 64.550 | <a href="mailto:jgonzalezcornet@itba.edu.ar">jgonzalezcornet@itba.edu.ar</a> |
| Juan Ignacio | Lategana        | 64.476 | <a href="mailto:jategana@itba.edu.ar">jategana@itba.edu.ar</a>               |
| Nicolás      | Priotto         | 64.663 | <a href="mailto:npriotto@itba.edu.ar">npriotto@itba.edu.ar</a>               |

72.39 - Autómatas, Teoría de Lenguajes y Compiladores

*Instituto Tecnológico de Buenos Aires*

Profesores: Ana María Arias Roig y Mario Agustín Golmar

22/06/2025

# Índice

|                                     |           |
|-------------------------------------|-----------|
| <b>1. Introducción.....</b>         | <b>1</b>  |
| <b>2. Modelo Computacional.....</b> | <b>2</b>  |
| 2.1 Dominio.....                    | 2         |
| 2.2 Lenguaje.....                   | 2         |
| <b>3. Implementación.....</b>       | <b>5</b>  |
| 3.1 Frontend.....                   | 5         |
| 3.2 Backend.....                    | 6         |
| 3.3 Dificultades encontradas.....   | 7         |
| <b>4. Futuras Extensiones.....</b>  | <b>8</b>  |
| <b>5. Conclusiones.....</b>         | <b>9</b>  |
| <b>7. Bibliografía.....</b>         | <b>10</b> |

# **1. Introducción**

En el siguiente trabajo se realizará un seguimiento de lo que fue el desarrollo de Synchro, un compilador que permite a los usuarios ingresar pseudocódigo relacionado a ejercicios de sincronización y concurrencia y recibir como output el código correspondiente en C para luego ser ejecutado. En este informe se realiza una descripción del dominio y el lenguaje abarcado y se detalla acerca de la implementación tanto en frontend como en backend. Para finalizar se mencionan posibles futuras extensiones a realizar en el compilador.

## 2. Modelo Computacional

### 2.1 Dominio

Desarrollar un lenguaje que permita especificar y simular ejercicios de sincronización con threads utilizando semáforos. El lenguaje debe permitir la declaración de semáforos y variables compartidas, la definición de threads, imprimir a salida estándar, generar tiempos de espera y la utilización de primitivas de sincronización como down y up.

El objetivo de este lenguaje es facilitar la prueba y validación de soluciones a problemas clásicos de concurrencia, como lectores-escritores, la exclusión mutua y la sincronización de threads. El grupo de usuarios esperado está compuesto por personas con conocimientos previos sobre sincronización, que comprenden los conceptos de concurrencia y paralelismo, así como el funcionamiento de los semáforos en C y poseen una gran comprensión del lenguaje. El código escrito en este DSL se traducirá a C, donde será retornado al usuario para poder ser compilado y ejecutado a fin de evaluar su correcto funcionamiento.

### 2.2 Lenguaje

Se trata de un lenguaje similar a C con facilidad para la resolución de ejercicios de sincronización. Veamos la sintaxis adicional que provee el lenguaje.

- Semáforos

```
// Declaración de un semáforo  
  
sem mutex = 1;  
  
// Decremento del valor del semáforo con espera bloqueante  
down(mutex);
```

```
// Incremento del valor del semáforo  
up(mutex);
```

- Threads

```
// Creación de n threads para una función  
thread(A,x); // Variable tipo int  
thread(B,5); // Constante
```

- Forever

```
// Generación de ciclo infinito (traducido a while(1))  
forever {  
    // Código ejecutado en cada iteración  
}
```

- Funciones propias del lenguaje

```
// Impresión a salida estándar  
print(a); // Variable tipo string  
print("hello world"); // Constante  
  
// Espera de x segundos  
sleep(a); // Variable tipo int  
sleep(5); // Constante
```

El resto de la sintaxis común, como por ejemplo declaración de variables, declaración de funciones, bloques condicionales (como if/else, while, for) y expresiones booleanas, mantienen el estilo del lenguaje C, con algunas pequeñas limitaciones en algunos casos. Estas limitaciones existentes se detallarán en la sección [4. Futuras Extensiones](#).

A continuación se provee un ejemplo más extenso y un posible caso de uso del lenguaje.

```
sem mutex = 1;
int sleepTime = 1;

int A() {
    forever {
        down(mutex);
        print("ACCESO SECCIÓN CRÍTICA - A");
        sleep(sleepTime);
        up(mutex);
        sleep(sleepTime);
    }
}

int B() {
    forever {
        down(mutex);
        print("ACCESO SECCIÓN CRÍTICA - B");
        sleep(sleepTime);
        up(mutex);
        sleep(sleepTime);
    }
}

int main() {
    int a = 2;
    thread(A, a);
    thread(B, 5);
    forever {}
    return 0;
}
```

## 3. Implementación

### 3.1 Frontend

El desarrollo del frontend comenzó con la implementación de *FlexPatterns.l* y *FlexActions.c*. En estos archivos se definen las palabras permitidas en el pseudocódigo y se realiza el pasaje de las mismas a “tokens” propios del compilador Flex/Bison para poder manejar las palabras luego. Es importante recordar que el input –como texto– es parseado únicamente por Flex: Bison nunca debe utilizar una función de parseo para releer el input, sino que se maneja con esos tokens.

Una vez completados estos archivos, se llevó a cabo la definición de la gramática en *BisonGrammar.y*. Aquí es donde se definió cómo se pueden componer esas expresiones, declaraciones, funciones, etc. entre sí.

Se definió que los programas comiencen con una “DeclarationList” que puede contener implicar la definición de funciones propias/asignación de variables y/o la definición de la función principal main (que debe ir última). Las funciones definidas por el usuario pueden recibir parámetros y utilizarlos para operar dentro de las mismas. Dentro de cualquier función, se pueden encontrar “Statements”, por ejemplo bloques de if, while, forever, for. Estos statements pueden ser abiertos o cerrados (si llevan llaves o no). Por otro lado, los statements pueden ser “SimpleStatements”, que vendrían a ser declaraciones de variables, asignaciones, o llamados a funciones propias provistas por nosotros creadores para esos usuarios (como print, sleep, up, down, thread).

Luego, se pueden encontrar “condition” y “expression”. Una condition expresa relaciones lógicas como and, or y not de expressions. Las expressions, permiten realizar operaciones sobre los tipos de datos, como suma, resta, incremento, etc. Permiten también operar con otras expressions permitiendo tener operaciones anidadas. Finalmente hay “types”, que representa los tipos de datos que posee

Synchro: int, string, float, boolean y sem; y también hay “constants”, que representan valores de estos tipos de datos.

## 3.2 Backend

El backend del compilador se encargó de procesar la información obtenida del árbol de sintaxis abstracta y generar un programa en C funcional a partir de ella. Esta parte se estructuró principalmente en torno a los módulos de manejo de ámbitos y símbolos, validación semántica, y generación de código.

En primer lugar, se implementaron *ScopeStack.c* y *SymbolTable.c*, que permiten registrar y mantener información sobre variables, funciones y su alcance. El sistema de scopes se basa en una pila que gestiona el contexto actual de ejecución (por ejemplo, cuando se entra o sale de una función o bloque), asegurando que los identificadores sean resueltos correctamente. Cada símbolo insertado se asocia a un identificador de scope, lo que permite borrar fácilmente símbolos de scopes anidados al salir de ellos. Además, en la tabla de símbolos se agregaron funciones y variables internas como print, sleep, up, down, thread, entre otras, con un scope reservado (-1), separadas de las definidas por el usuario.

Luego, el módulo *TypeChecking.c* realiza la validación semántica del programa, verificando que todas las expresiones, asignaciones, declaraciones, condiciones y llamadas a funciones sean válidas en cuanto a tipos de datos. Por ejemplo, asegura que no se pueda asignar un float a un string, que los operadores aritméticos se usen sobre tipos numéricos, o que los incrementos y decrementos se apliquen sólo sobre variables adecuadas. Esta validación se hace de manera inmediata al parsear, y se realiza consultando la tabla de símbolos y los tipos esperados definidos por el lenguaje.



Finalmente, el archivo *Generator.c* se encarga de traducir el AST a un programa en C compilable. Esto incluye generar los headers necesarios (*stdio.h*, *pthread.h*, etc.), mapear tipos del lenguaje a tipos de C (*int*, *char\**, *bool*, etc.), y emitir declaraciones y sentencias en el orden correcto. Las funciones definidas por el usuario se traducen a funciones de tipo *void\** compatibles con hilos de *pthread*. Además, se renombra la función *main* del usuario para evitar conflictos con el *main* del runtime generado automáticamente. Este runtime (*SynchronizationRuntime.c*) inicializa el entorno de ejecución, gestiona la impresión y sincronización mediante mutexes, y genera funciones auxiliares que simulan el comportamiento de primitivas como *sleep* o *thread*. Así, se garantiza que el código en C conserve la semántica del pseudocódigo de entrada.

En resumen, el backend transforma el AST en un programa en C funcional y seguro, gestionando correctamente los scopes, tipos y estructuras del lenguaje, y permitiendo que el programa generado respete las restricciones de sincronización y concurrencia definidas por el compilador.

### **3.3 Dificultades encontradas**

La principal problemática en el desarrollo de este trabajo se presentó a la hora de crear la gramática de Bison. En la misma, se encontraron muchos conflictos, principalmente debido al problema de “dangling else”. Tras un tiempo de investigación, se debió reformular el uso de los *ClosedStatements* y *OpenStatements*, a la vez que separar las *Expressions* y *Conditions*.

Otro problema presentado fueron los memory leaks que se veían al ejecutar los tests. Mediante un proceso de debugging se lograron encontrar las zonas de memoria que eran reservadas pero no liberadas y mediante la modificación de las *BisonActions.c* y el *AbstractSyntaxTree.c* se lograron eliminar dichos problemas de manejo de memoria dinámica.

## 4. Futuras Extensiones

Actualmente no hay expresiones *booleanas* anidadas en *synchro* ya que *condition* no permite comparar con otra *expression*. Es decir, solo se puede tener un *and*, *or* o *not* por *condition*. Para traspasar esto hay que crear variables auxiliares. En un futuro se espera poder anidar *conditions* para realizar lógica booleana compleja en una sola línea.

Los *return* aceptados en este momento son una variable o una constante. En una próxima versión se espera la implementación de *return* complejos, que permitan devolver condiciones o expresiones.

Los *for* loops aceptan un solo statement en la declaración y en la acción a realizar, se puede agregar la aceptación de más de un statement separados con una coma.

No se pueden concatenar strings, en un futuro se espera poder concatenarlas fácilmente para por ejemplo utilizar *print*, que ahora solo recibe una sola constante o variable de tipo string. Además en un futuro se espera que también pueda imprimir variables de otros tipos.

No se pueden crear variables no *inicializadas* con un valor aún. Se espera poder solamente declarar una variable y luego darle un valor en runtime en una próxima versión.

Al usar threads, hay que poner un *forever* si no quiere que **todos mueran** cuando finaliza la función *main*. Para esto, se puede también implementar un sistema de procesos independientes con IPC para evitar esta dependencia del proceso padre que poseen los threads en linux.

## 5. Conclusiones

Utilizando herramientas como Flex y Bison se ha logrado crear un compilador en C para poder cumplir con las expectativas del lenguaje presentado. Mediante el mismo, se pudieron representar distintos casos de sincronización de threads y ejecutarlos en nuestras computadoras con Linux.

En la etapa de frontend se logró comprender en profundidad cómo crear una gramática sin conflictos combinando la teoría cubierta en la materia con la práctica. Se han encontrado dificultades en ese proceso pero las mismas permitieron llegar a una gramática más simplificada y clara.

En la etapa de backend se logró implementar la generación de código agregada a implementaciones como la tabla de símbolos y el stack de scopes. En la misma se logró comprender cómo los compiladores pueden “traducir” código entre 2 lenguajes distintos, que ha sido una buena adición a lo visto en la teoría de la materia.

## 6. Bibliografía

1. Free Software Foundation. (n.d.). *Bison Manual: Shift/Reduce Conflicts*. GNU Project. Recuperado de:  
[https://www.gnu.org/software/bison/manual/html\\_node/Shift\\_002fReduce.html](https://www.gnu.org/software/bison/manual/html_node/Shift_002fReduce.html)
2. Free Software Foundation. (n.d.). *Bison Manual: Non-Operators*. GNU Project. Recuperado de:  
[https://www.gnu.org/software/bison/manual/html\\_node/Non-Operators.html](https://www.gnu.org/software/bison/manual/html_node/Non-Operators.html)
3. Wikipedia. (2024). *Dangling else*. Wikipedia, The Free Encyclopedia. Recuperado de: [https://en.wikipedia.org/wiki/Dangling\\_else](https://en.wikipedia.org/wiki/Dangling_else)
4. GeeksforGeeks. (2023). *Dangling else ambiguity in Compiler Design*. Recuperado de:  
<https://www.geeksforgeeks.org/compiler-design/dangling-else-ambiguity/>